SMART RESEARCH PAPER RECOMMENDATION SYSTEM USING ETCM FRAMEWORK

Explainable Topic-Coverage Matching using Rabin–Karp,
Edit Distance, and Bipartite Graph Matching

A Project Based Learning Report submitted in partial fulfilment of the requirements for the award of the degree of

Bachelor of Technology

in Computer Science and Engineering

ADVANCED DATA STRUCTURES AND ALGORITHMS

Course Code: [Course Code]

Submitted by

2510030315 — V SATYA SHIVA SIDDHARTH
2510030320 — BONTHAPALLY ABHILASH
2510030334 — C CHETAN
2510030354 — MOHHAMAD ABDUL REHYAN

Under the guidance of

[Faculty Guide Name and Designation]



Department of Computer Science and Engineering

Koneru Lakshmaiah Education Foundation, Aziz Nagar

Aziz Nagar – 500075

2026

Abstract

ETCM (Explainable Topic-Coverage Matching) is a Java desktop application that recommends research papers for a text query or for supplied paper text using classical data structures and algorithms rather than machine learning. The application preprocesses text, removes stop words and extracts technical topics, including predefined multi-word phrases. An inverted index retrieves candidate papers, Rabin–Karp string matching checks exact topic occurrences, Edit Distance computed by dynamic programming measures approximate string similarity, and a topic-coverage measure records how many query topics appear in each paper. These components are combined into an Explainable Recommendation Score (ERS) defined as ERS = 0.5E + 0.3A + 0.2C, where E is the exact score, A the approximate score and C the coverage score.

Papers are ranked by ERS in descending order. A topic–paper bipartite graph and a maximum bipartite matching based on augmenting paths then assign distinct papers to distinct topics where eligible edges exist. The ranking, the coverage calculation and the one-to-one matching are separate: the matching does not maximise coverage or ERS and is not a set-cover algorithm. The interface is written in JavaFX 21.0.12 on Java 21, paper metadata is loaded from CSV files, and recommendations can be exported as CSV and text reports.

The project is a prototype for demonstrating practical uses of hashing, dynamic programming, indexing and graph matching in an explainable recommendation setting. In the source version described for this report, the exact score and the coverage score are computed by the same method and are therefore identical; this is documented as a limitation. ETCM depends on the supplied dataset and on heuristic, rule-based topic extraction, does not understand full semantics, and does not establish the quality or truthfulness of any paper. All worked examples are hypothetical, no test outcome or benchmark is claimed, and items that require verification against the source code are labelled.

Keywords:
Research paper recommendation, explainable recommendation, topic coverage, inverted index, Rabin–Karp algorithm, Edit Distance, dynamic programming, bipartite matching, Java, JavaFX.

List of Figures

Figure	Title
Figure 1	System architecture
Figure 2	Class relationships
Figure 3	Data flow diagram
Figure 4	End-to-end workflow
Figure 5	Inverted index example
Figure 6	Edit Distance dynamic-programming example
Figure 7	Recommendation scoring pipeline
Figure 8	Bipartite topic-paper graph (illustrative)
Figure 9	Main window with query input (screenshot placeholder)
Figure 10	Ranked recommendations (screenshot placeholder)
Figure 11	Bipartite matching view (screenshot placeholder)
Figure 12	Exported report (screenshot placeholder)

List of Tables

Table	Title
Table 1	Project objectives and where they are realised
Table 2	Functional requirements
Table 3	Non-functional requirements
Table 4	Technology requirements and roles
Table 5	Software components and responsibilities
Table 6	Workflow stages and responsible components
Table 7	Rabin–Karp worked example (pattern "ab", text "xabc")
Table 8	Components of the Explainable Recommendation Score
Table 9	Complexity analysis
Table 10	Testing layers
Table 11	Test cases: preprocessing and topic extraction
Table 12	Test cases: Rabin–Karp and Edit Distance
Table 13	Test cases: coverage, scoring, ranking and matching
Table 14	Test cases: CSV input, export and interface
Table 15	Test execution status
Table 16	Implemented capabilities as described and evidence status
Table 17	Hypothetical example: query topics and papers
Table 18	Hypothetical example: component scores, ERS and ranking
Table 19	Hypothetical example: augmenting-path matching trace
Table 20	Evaluation against objectives
Table 21	Limitations and future enhancements
Table 22	Technical source map
Table 23	Items requiring verification before submission
Table 24	Illustrative paper CSV fields

Table of Contents

Section	Coverage
Introduction	Problem context, objectives, scope, related concepts and requirements
METHODOLOGY	Architecture, workflow, algorithms and data structures, implementation details and complexity analysis
EXPERIMENTS	Test strategy, formal test plan and reproducibility
RESULTS	Implemented capabilities as described, a hypothetical worked example, advantages and evaluation
CONCLUSION and FUTURE WORK	Limitations, ethical considerations, future enhancements and conclusion
References	Algorithm, language and information-retrieval references
Technical source map	Topic-to-artefact mapping and items requiring verification
Appendices	A: Pseudocode; B: Sample dataset format; C: User manual; D: Viva questions and answers

The report uses the major section headings from the supplied academic template. Subsections organise the technical explanation. Statements about the software are taken from the project description supplied by the team; the source code was not available while this report was prepared, so items to be confirmed against it, and all example values, are labelled as such.

List of Abbreviations

Abbreviation	Expansion
API	Application Programming Interface
CSV	Comma-Separated Values
DFS	Depth-First Search
DP	Dynamic Programming
DSA	Data Structures and Algorithms
ERS	Explainable Recommendation Score
ETCM	Explainable Topic-Coverage Matching
GUI	Graphical User Interface
IDE	Integrated Development Environment
IR	Information Retrieval
JDK	Java Development Kit
RK	Rabin–Karp
UI	User Interface

Introduction

Background and motivation

Researchers and students searching a collection of papers usually begin with a keyword query. A plain keyword match can return a paper that mentions only one part of a multi-topic query, for example a paper on machine learning that never mentions medical imaging or data privacy. It also gives little explanation of why a paper was returned or how well it covers what the user asked for.

ETCM addresses this with classical data structures and algorithms. It retrieves candidates with an inverted index, checks exact occurrences of topics with Rabin–Karp string matching, measures approximate string similarity with Edit Distance, calculates topic coverage, combines the measures into a score and displays how topics relate to papers through a bipartite matching. Each step is deterministic and can be inspected, which supports the project's emphasis on transparent decisions over opaque learned models [4], [5].

Problem statement

Simple keyword searches can return papers that match only one part of a multi-topic query. Users need a way to compare research papers using exact matching, approximate matching and topic coverage, while also understanding why each paper is recommended.

> **Research question: How can a research paper recommendation system use classical data structures and algorithms to retrieve relevant papers, evaluate topic relevance, and explain its recommendations?**

Project objectives

Table 1 lists the fifteen objectives of the project and the component that realises each, according to the project description.

Table 1. Project objectives and where they are realised

No.	Objective	Realised by (as described)
1	Build a Java desktop application for research paper recommendation	Main, ETCMApplication
2	Accept text queries and uploaded-paper text as inputs	ETCMApplication, TextPreprocessor
3	Normalise text and extract useful technical topics	TextPreprocessor
4	Recognise predefined multi-word technical phrases	TextPreprocessor
5	Use an inverted index for candidate paper retrieval	InvertedIndex
6	Implement Rabin–Karp string matching	RabinKarp
7	Implement Edit Distance using dynamic programming	EditDistance
8	Calculate topic coverage for each candidate paper	TopicCoverage
9	Compute an Explainable Recommendation Score (ERS)	RecommendationEngine
10	Rank recommendations in descending score order	RecommendationEngine
11	Build a topic–paper bipartite graph	BipartiteMatcher, Recommendation
12	Apply maximum bipartite matching for distinct topic–paper assignments	BipartiteMatcher
13	Provide a JavaFX interface for displaying results	ETCMApplication, BipartiteGraphView
14	Load research paper metadata from CSV files	PaperCsvReader, Paper
15	Export recommendations in CSV and text-report formats	ReportExporter

Scope and boundaries

ETCM is a prototype intended to demonstrate practical applications of data structures and algorithms in information retrieval and recommendation. It runs locally on a modest dataset supplied as CSV files. The project explicitly:

- does not use deep learning or transformer models and does not depend on a trained machine-learning model;
- uses deterministic, explainable algorithms;
- depends on the quality and completeness of the supplied dataset;
- uses predefined phrases and heuristic topic extraction;
- does not establish the scientific quality or truthfulness of a research paper;
- does not automatically understand the full semantic meaning of a paper;
- may not identify synonyms or conceptual relationships reliably, and uses approximate string similarity rather than semantic similarity.

No claim is made that ETCM outperforms established recommendation systems; that would require experimental evidence that has not been collected.

Background and related concepts

Information retrieval. An inverted index maps each term to the documents that contain it, so that candidates for a query can be found without scanning every document [5]. Retrieval by shared terms does not measure meaning: it finds papers that contain the words, not necessarily papers about the idea.

String matching. The Rabin–Karp algorithm compares a hash of the pattern with a rolling hash of each text window and verifies a character-by-character match only when hashes agree [1], [4]. **Approximate matching.** Edit Distance counts the insertions, deletions and replacements needed to turn one string into another and is computed by dynamic programming [2], [3], [4]. **Graph matching.** In a bipartite graph a matching is a set of edges that share no vertex; augmenting paths enlarge a matching until it is maximum [4], [6].

Explainability. Here, explainable means that every score component can be traced to a specific step: which topics were found, which similarity was computed and which weights were applied. It does not mean that the recommendation is correct or has been validated.

Functional requirements

Table 2 lists the functional requirements. The status column records what the project description says; it does not certify that a feature has been tested.

Table 2. Functional requirements

ID	Requirement	Status grounded in project description
FR-01	Accept a user query	Described as implemented
FR-02	Accept uploaded-paper text, according to supported application functionality	Described; exact input mechanism to be confirmed
FR-03	Extract topics (predefined phrases and useful words)	Described as implemented
FR-04	Load paper metadata from CSV files	Described as implemented
FR-05	Retrieve candidate papers using an inverted index	Described as implemented
FR-06	Calculate exact and approximate matching scores	Described; exact score shares its calculation with coverage
FR-07	Calculate topic coverage and ERS	Described as implemented
FR-08	Rank results by ERS in descending order	Described as implemented
FR-09	Display the topic–paper bipartite matching	Described as implemented; edges are not drawn explicitly
FR-10	Export reports in CSV and text formats	Described as implemented

Non-functional requirements

Table 3 states the quality attributes the design aims for. No enterprise-grade scalability, security certification or performance guarantee is claimed.

Table 3. Non-functional requirements

Attribute	Requirement
Usability	A JavaFX interface that shows ranked recommendations, matching information and summary statistics.
Modularity	Separate classes for preprocessing, indexing, matching, scoring, graph matching, input, output and interface.
Maintainability	Small single-purpose classes so a component can be changed or tested on its own.
Explainability	Every score component and matched topic can be traced to a specific algorithmic step.
Deterministic behaviour	The same dataset and query produce the same recommendations; no randomness or trained model is involved.
Reasonable performance	Acceptable response on a modest local dataset; no timing figure is claimed because none has been measured.
Data integrity	CSV loading should handle headers, quoted fields and incomplete rows without corrupting the loaded collection.

Technology requirements

Table 4 lists the technologies. JavaFX provides the graphical interface, while the data-structure and algorithm logic is implemented in plain Java [7], [8], [9].

Table 4. Technology requirements and roles

Item	Details	Role in the project
Java	Java 21	Implementation language for all algorithms and the application
JavaFX	21.0.12	Graphical interface (windows, controls, result display)
Eclipse IDE	Eclipse	Development environment
Java Collections Framework	HashMap, LinkedHashSet, lists	Inverted index, candidate sets and result lists
CSV input	Paper metadata files	Source of the paper collection
CSV and text output	Report files	Exported recommendations

Assumptions and constraints

The system assumes a supplied CSV dataset with paper identifiers, titles, authors, keywords and abstracts, and a machine able to run Java 21 with JavaFX 21.0.12. Topic extraction depends on a predefined phrase list and simple rules. Matching is textual: it compares characters, not meanings. These constraints keep the system explainable and suitable for demonstrating the algorithms.

Success criteria

The prototype is considered functionally successful when its core flow can be reproduced: a query or paper text is normalised into topics; candidates are retrieved from the loaded dataset; exact, approximate and coverage measures are computed; papers are ranked by ERS; the matching is displayed; invalid or empty input is handled without a crash; and a report can be exported. No success criterion in this report relies on a measured accuracy figure.

Why the project matters for data structures and algorithms

The project brings several taught techniques into one working pipeline: hash maps and sets for indexing, rolling hashing for string search, dynamic programming for approximate similarity, sorting for ranking and augmenting paths for graph matching. Studying them together shows how the cost of each step depends on different variables, and how a practical system must state its assumptions when it reports complexity.

METHODOLOGY

Development approach

ETCM is organised as a modular Java project of fourteen classes, each with a narrow responsibility: data classes for papers and recommendations, one class per algorithm, an engine that coordinates scoring, a matcher for the bipartite graph, input and output classes, and the JavaFX interface. This separation keeps user-interface concerns away from algorithm code and lets each algorithm be tested on its own. Figure 1 shows the layers; the class list follows the project description, and method-level behaviour is to be confirmed against the source.

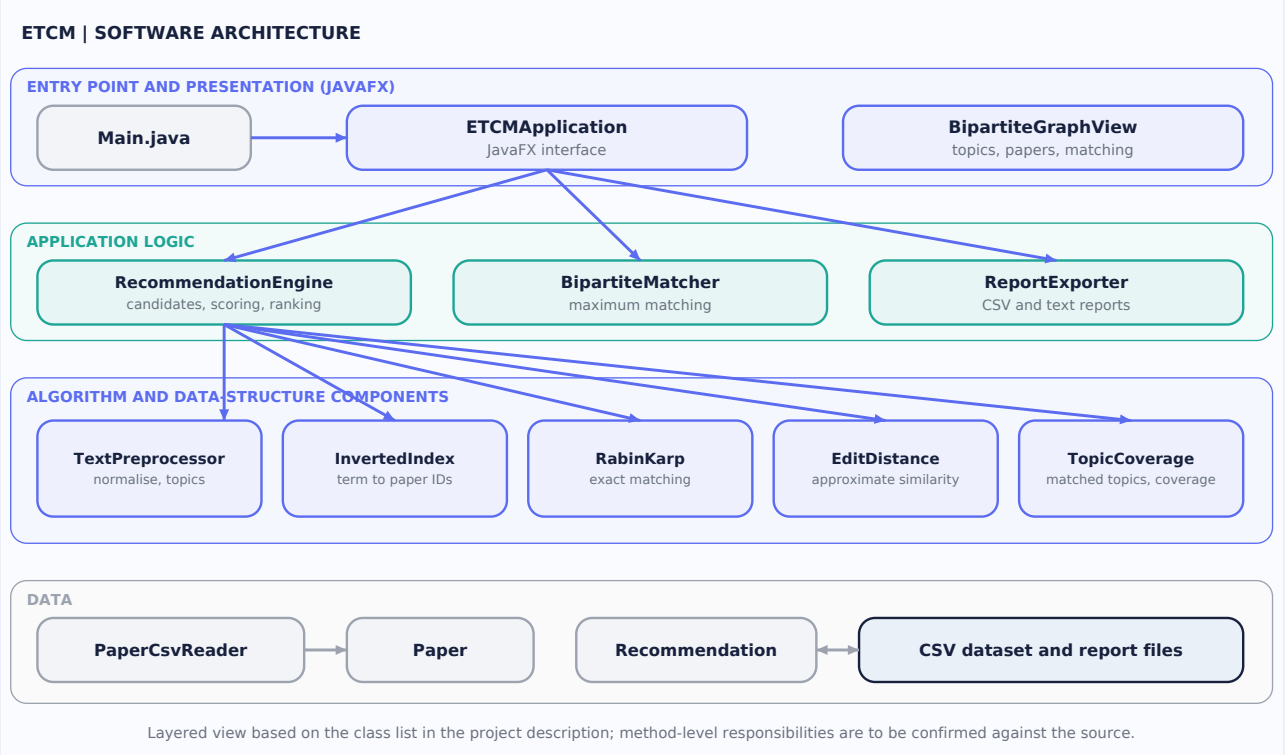


Figure 1. Software architecture of ETCM. Components are grouped by role; the DSA logic is plain Java and JavaFX supplies the interface.

Software components and responsibilities

Table 5 lists each class with the responsibility given in the project description. Nothing is attributed to a class beyond that description.

Table 5. Software components and responsibilities

Class	Responsibility (as described)	Interacts with
Paper	Stores paper ID, title, authors, keywords and abstract	PaperCsvReader, InvertedIndex, RecommendationEngine
TextPreprocessor	Normalises text and extracts topics	RecommendationEngine, InvertedIndex
InvertedIndex	Indexes terms and retrieves candidate papers	Paper, RecommendationEngine
RabinKarp	Implements exact string matching	RecommendationEngine
EditDistance	Calculates edit distance and normalised similarity	RecommendationEngine
TopicCoverage	Identifies matched topics and calculates coverage	RecommendationEngine
Recommendation	Stores paper scores and matched topics	RecommendationEngine, BipartiteMatcher, ReportExporter
RecommendationEngine	Retrieves candidates, scores them and ranks recommendations	All algorithm classes, Recommendation
BipartiteMatcher	Performs maximum bipartite matching	Recommendation, BipartiteGraphView
BipartiteGraphView	Displays topics, papers, matching relationships and summary labels	ETCMApplication, BipartiteMatcher
PaperCsvReader	Loads paper data from CSV	Paper, ETCMApplication
ReportExporter	Exports CSV and text reports	Recommendation, ETCMApplication
ETCMApplication	Manages the JavaFX application interface	All user-facing classes
Main	Application entry point	ETCMApplication

How the components interact

Main starts ETCMApplication. When a dataset is loaded, PaperCsvReader creates Paper objects and the inverted index is built from them. For a query, TextPreprocessor produces topics; RecommendationEngine asks InvertedIndex for candidates and then uses RabinKarp, EditDistance and TopicCoverage to score each candidate, storing the result in Recommendation objects that it sorts. BipartiteMatcher works from the matched-topic lists in those recommendations, BipartiteGraphView presents the topics, papers and matching, and ReportExporter writes the recommendations to files. Figure 2 shows these dependencies and Figure 3 shows the data flow.

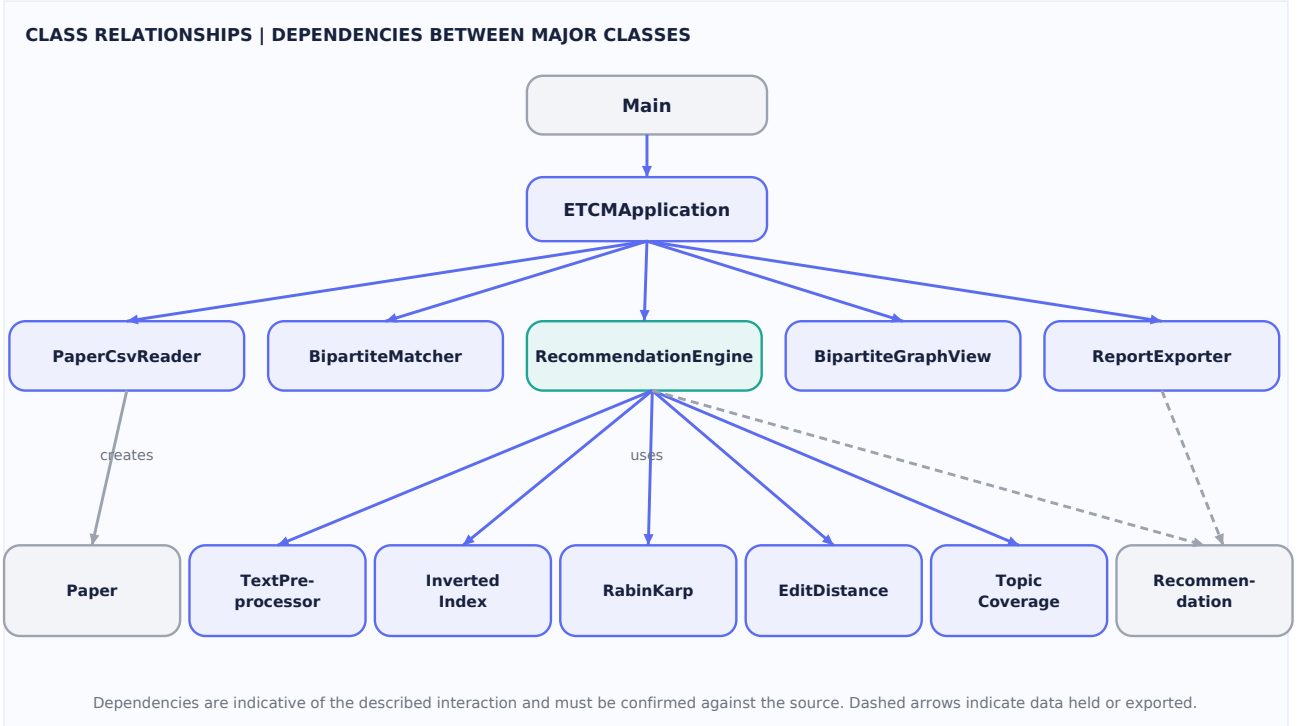


Figure 2. Class relationships. Arrows show the indicative direction of use; they are to be confirmed against the source.

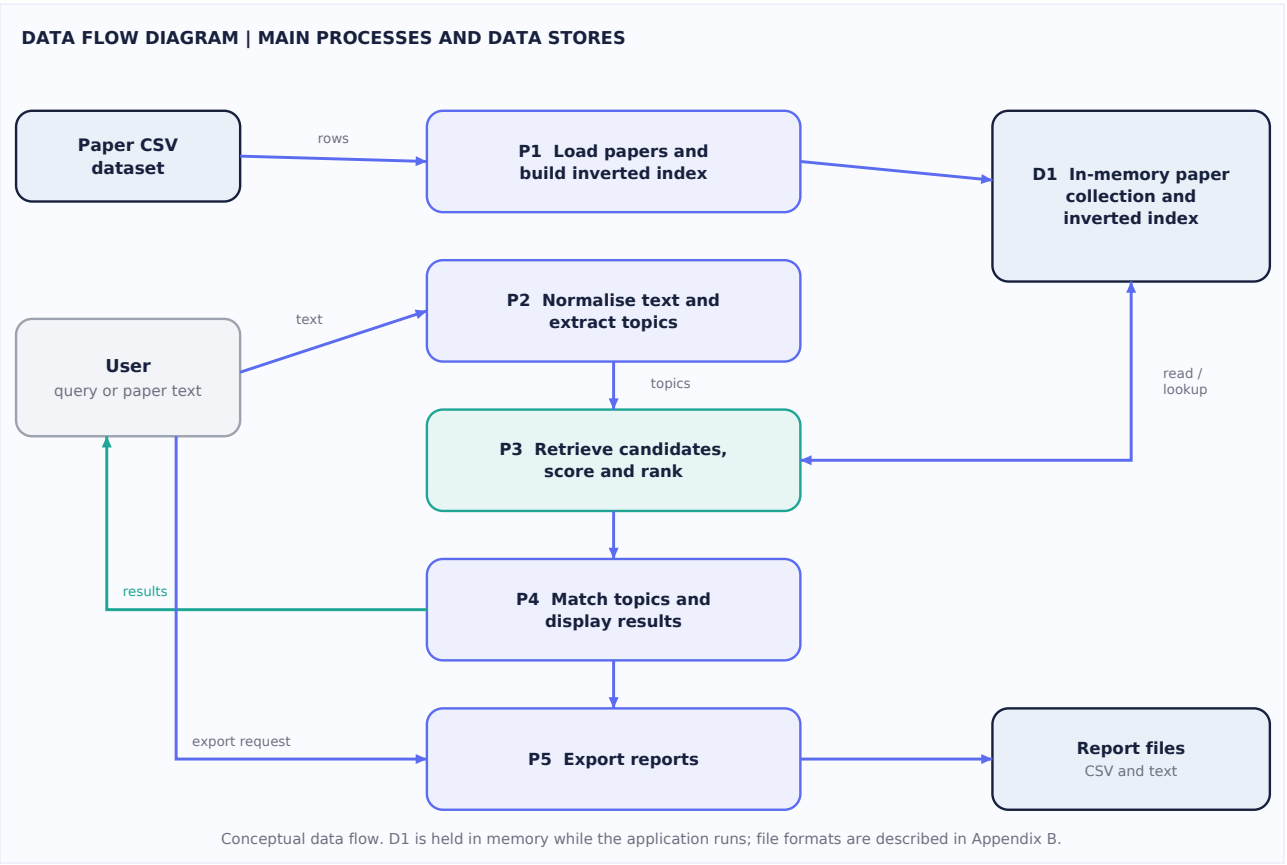


Figure 3. Data flow diagram with five processes, one in-memory data store and the report files.

System workflow

Figure 4 shows the thirteen stages of the workflow and Table 6 explains each stage and the component responsible for it.

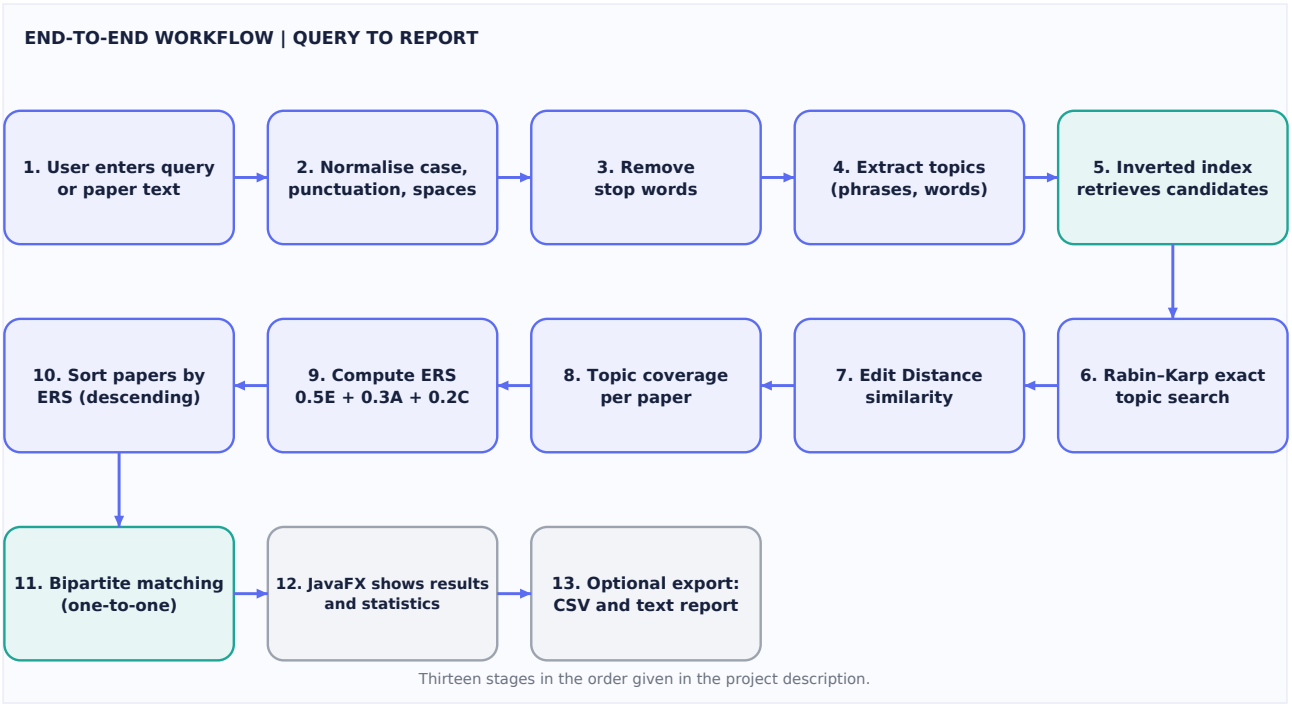


Figure 4. End-to-end workflow from user input to exported report.

Table 6. Workflow stages and responsible components

Step	Stage	Component	Explanation
1	Input	ETCMApplication	The user enters a research query or supplies paper text.
2	Normalisation	TextPreprocessor	Case is lowered, punctuation is replaced and whitespace is collapsed.
3	Stop-word filtering	TextPreprocessor	Common words such as "for" and "and" are removed.
4	Topic extraction	TextPreprocessor	Predefined technical phrases and useful individual words become topics.
5	Candidate retrieval	InvertedIndex	Indexed terms identify papers that may be relevant.
6	Exact matching	RabinKarp	Exact occurrences of topics in paper text are checked.
7	Approximate similarity	EditDistance	Normalised edit-distance similarity is calculated.
8	Topic coverage	TopicCoverage	The fraction of query topics present in each paper is measured.
9	Scoring	RecommendationEngine	ERS combines the implemented scoring components.
10	Ranking	RecommendationEngine	Candidates are sorted by ERS in descending order.
11	Matching	BipartiteMatcher	Distinct papers are assigned to topics where eligible edges exist.
12	Display	ETCMApplication, BipartiteGraphView	Recommendations, matching information and summary statistics are shown.
13	Export	ReportExporter	The user can export CSV and text reports.

Algorithms and data structures

The following subsections describe each component. Their order follows the workflow: preprocessing, indexing, exact matching, approximate matching, coverage, scoring and graph matching.

A. Text preprocessing and topic extraction

Preprocessing applies three normalisation steps. Lowercasing makes matching case-insensitive; punctuation is replaced so that "data-privacy," and "data privacy" are treated alike; and whitespace is collapsed so that runs of spaces become one. Stop-word removal then drops common words that carry little topical meaning. Topic extraction recognises predefined multi-word technical phrases and keeps useful individual words.

For example, the query "Machine-Learning for Medical Imaging, and Data Privacy!" becomes "machine learning for medical imaging and data privacy" after normalisation, and "machine learning medical imaging data privacy" after the stop words "for" and "and" are removed. If "machine learning", "medical imaging" and "data privacy" are in the predefined phrase list, they become three topics. Whether single words inside a recognised phrase are also kept as separate topics is to be confirmed.

The approach is rule-based and has clear limits: a technical phrase that is not in the predefined list is only found as individual words; synonyms and abbreviations ("ML" for "machine learning") are not related; the stop-word list affects what is kept; and punctuation replacement can split terms that should stay together. These limits are reasons for the approximate-matching and coverage steps, not problems the preprocessor solves.

B. Inverted index

The inverted index maps each term to the set of paper identifiers that contain it. A HashMap from term to LinkedHashSet of identifiers is used: the HashMap gives expected constant-time lookup and insertion, and the LinkedHashSet stores each identifier once while keeping insertion order, which makes retrieval output deterministic [9]. Figure 5 shows an example with the terms and identifiers machine → {P001, P004}, learning → {P001, P002} and imaging → {P003, P004}.

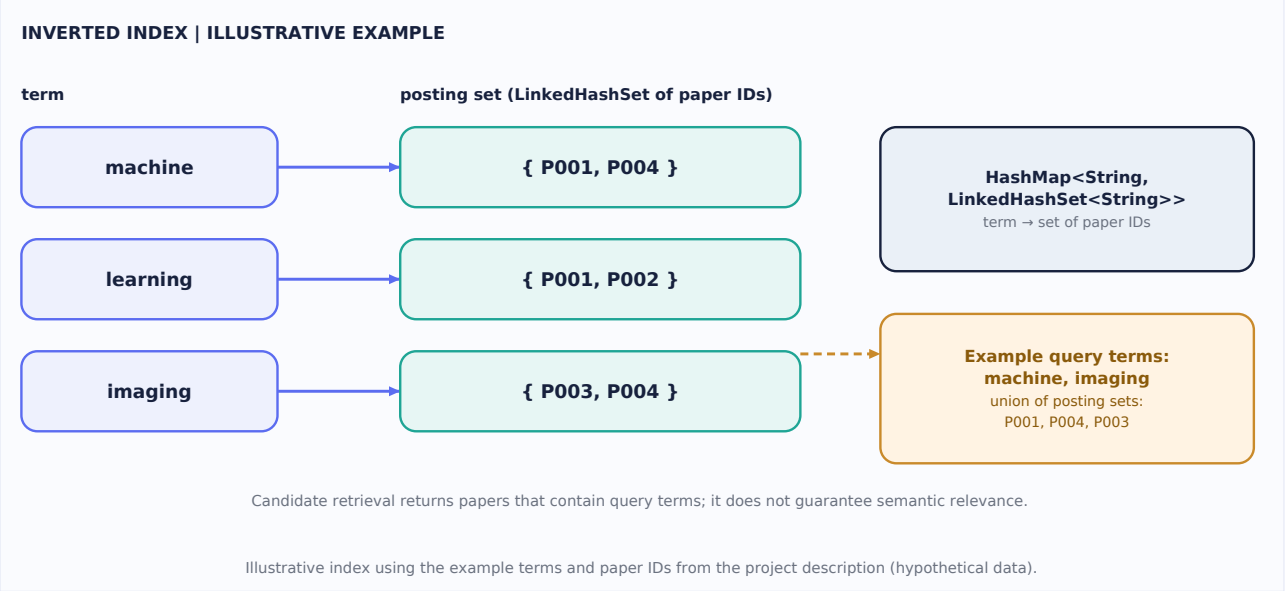


Figure 5. Inverted index example (hypothetical data). A query for machine and imaging retrieves P001, P004 and P003.

Under the assumption that hash operations take expected constant time, building the index takes time proportional to the number of indexed tokens, and retrieving candidates for a query takes time proportional to the sizes of the posting sets involved. The index only finds papers that contain the query terms. It does not guarantee semantic relevance, and a paper using different words for the same idea will not be retrieved. Which paper fields are indexed (title, keywords, abstract) and whether candidates are the union of all term sets are to be confirmed from the source.

C. Rabin–Karp string matching

Rabin–Karp searches for a pattern of length m in a text of length n by comparing numbers instead of strings. The hash of the pattern and the hash of each length-m window of the text are computed; when two hashes are equal, the characters are compared one by one to confirm the match. The Java implementation uses BASE = 256 and PRIME = 101 [1]. The pattern hash is

$$H(P) = (P[0]\cdot BASE^{m-1} + P[1]\cdot BASE^{m-2} + \ldots + P[m-1]) \bmod PRIME \quad (1)$$

and the rolling hash moves from the window starting at s to the window starting at s+1 by removing the leading character and appending the next one, where $h = BASE^{m-1} \bmod PRIME$:

$$t_{s+1} = (BASE\cdot(t_s - T[s]\cdot h) + T[s+m]) \bmod PRIME \quad (2)$$

If the result is negative, PRIME is added so the value stays in the range 0 to PRIME − 1. Because a small PRIME makes unequal strings occasionally share a hash (a collision), every hash match is verified character by character, so a collision causes extra work but never a wrong match.

```
RABIN-KARP(text T, pattern P)            BASE = 256, PRIME = 101
  n <- length(T);  m <- length(P)
  if m = 0 or m > n then return NO-MATCH          (empty pattern: to be confirmed)
  h <- BASE^(m-1) mod PRIME
  p <- 0;  t <- 0
  for i <- 0 to m-1 do
      p <- (BASE*p + P[i]) mod PRIME
      t <- (BASE*t + T[i]) mod PRIME
  for s <- 0 to n-m do
      if p = t then
          if T[s .. s+m-1] = P then return s          (verified match)
      if s < n-m then
          t <- (BASE*(t - T[s]*h) + T[s+m]) mod PRIME
          if t < 0 then t <- t + PRIME
  return NO-MATCH
```

Table 7 traces the algorithm for the pattern "ab" in the text "xabc", using the character codes x = 120, a = 97, b = 98 and c = 99. The pattern hash is H("ab") = (97·256 + 98) mod 101 = 84. The first window "xa" has hash 12, so it is skipped; rolling to "ab" gives (256·(12 − 120·54) + 98) mod 101 = 84 after correcting a negative value, the hashes agree, and verification confirms a match at index 1. The final window "bc" has hash 38.

Table 7. Rabin–Karp worked example (pattern "ab", text "xabc")

Window start	Window	Window hash	Equals pattern hash 84?	Action
0	xa	12	No	Skip; roll the hash
1	ab	84	Yes	Verify characters: "ab" = "ab", match at index 1
2	bc	38	No	End of text

A collision can be shown with the same parameters: "ju" also has hash 84 ((106·54 + 117) mod 101 = 84), so searching for "ab" in "ju" finds equal hashes, but the character check fails and no match is reported.

Complexity. With n the text length and m the pattern length, Rabin–Karp takes expected O(n + m) time under favourable assumptions, namely few spurious hash matches, and O(nm) time in the worst case when many windows collide and each is verified. It uses O(1) additional space. This is the cost of the string-search component only, not of the whole recommendation pipeline.

D. Edit Distance

Edit Distance is the smallest number of single-character insertions, deletions and replacements needed to turn one string into another [2], [3]. For strings of lengths a and b, a table D has (a + 1) rows and (b + 1) columns, where D[i][j] is the distance between the first i characters of the first string and the first j characters of the second. The first row and column are D[i][0] = i and D[0][j] = j. The other cells use

$$D[i][j] = \min(\, D[i-1][j] + 1,\ D[i][j-1] + 1,\ D[i-1][j-1] + \text{cost}\,) \quad (3)$$

where cost is 0 if the characters match and 1 otherwise. The distance is D[a][b]. To compare strings of different lengths on one scale, the distance is normalised:

$$\text{similarity} = 1 - \text{distance} / \max(\text{length}_1, \text{length}_2) \quad (4)$$

Empty strings need care. If one string is empty the distance equals the length of the other, so the similarity is 0. If both are empty, $\max(\text{length}_1, \text{length}_2)$ is 0 and the formula would divide by zero; a guard is needed, and the usual convention is a similarity of 1.0. The exact handling in the source is to be verified.

```
EDIT-DISTANCE(X, Y)
  a <- length(X);  b <- length(Y)
  create table D[0..a][0..b]
  for i <- 0 to a do  D[i][0] <- i
  for j <- 0 to b do  D[0][j] <- j
  for i <- 1 to a do
      for j <- 1 to b do
          cost <- 0 if X[i] = Y[j] else 1
          D[i][j] <- min(D[i-1][j] + 1, D[i][j-1] + 1, D[i-1][j-1] + cost)
  return D[a][b]

SIMILARITY(X, Y)
  m <- max(length(X), length(Y))
  if m = 0 then return 1.0                          (convention; to be verified)
  return 1 - EDIT-DISTANCE(X, Y) / m
```

Figure 6 shows the table for "graph" and "grape". The strings differ only in the last character, so the distance is 1 (replace h with e) and the similarity is 1 − 1/5 = 0.8. The full table uses O(ab) time and O(ab) space; a version that keeps only two rows could reduce space, but that is not claimed for the project.

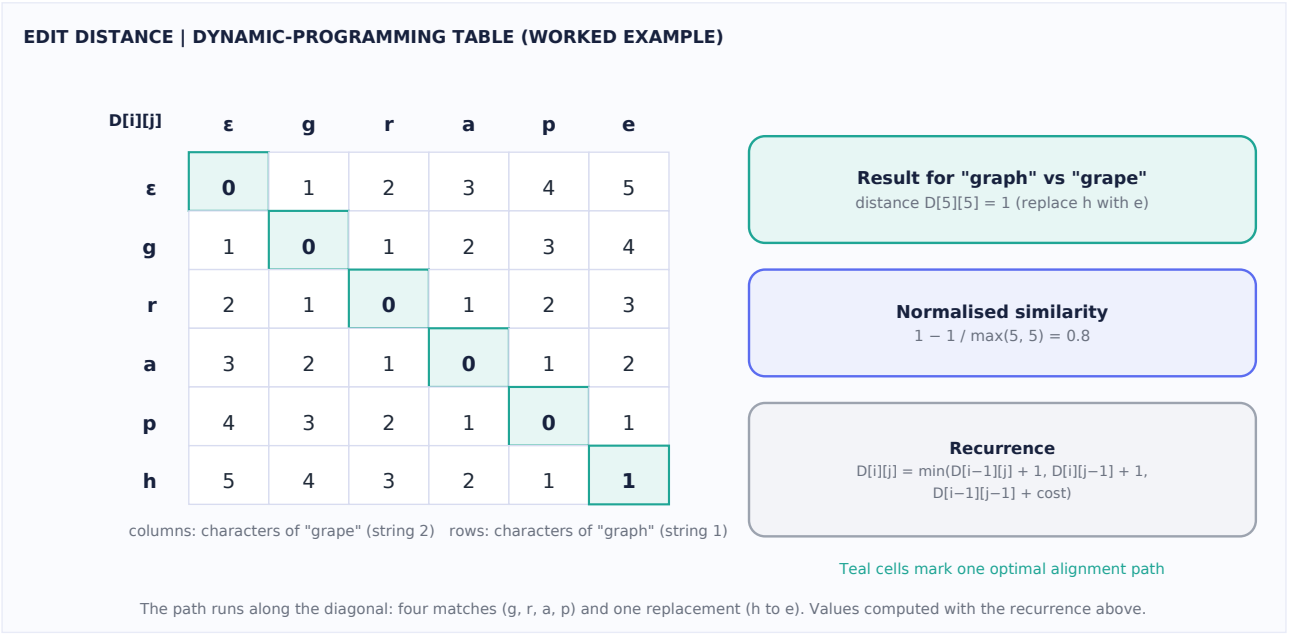


Figure 6. Dynamic-programming table for the Edit Distance of "graph" and "grape". The shaded diagonal is one optimal alignment.

E. Topic coverage

Topic coverage measures how many of the extracted query topics appear in a paper. In the implemented definition,

$$\text{Coverage} = (\text{number of extracted query topics found in the paper}) / (\text{total number of extracted query topics}) \quad (5)$$

If no topics were extracted, the current implementation returns 0, which avoids division by zero. A topic counts as found when its normalised phrase occurs exactly in the normalised paper text. A paper that contains two of three topics therefore has coverage 2/3, and the list of found topics is stored with the recommendation so the result can be explained.

F. Explainable Recommendation Score (ERS)

The Explainable Recommendation Score combines three components with fixed weights:

$$ERS = 0.5E + 0.3A + 0.2C \quad (6)$$

Table 8 defines the components. The weights 0.5, 0.3 and 0.2 are design choices made for this project; they are not scientifically established universal values, and a different weighting could be preferable for another dataset or purpose.

Table 8. Components of the Explainable Recommendation Score

Component	Weight	Meaning	Source in the described version
E (exact score)	0.5	Strength of exact topic matching	TopicCoverage.calculate()
A (approximate score)	0.3	Approximate string similarity	Edit Distance based similarity
C (coverage score)	0.2	Fraction of query topics found	TopicCoverage.calculate()

Implementation accuracy note: in the supplied code version, exactScore and coverageScore are both calculated by TopicCoverage.calculate(), so E and C are currently identical. They are not independent measures. With E = C the score reduces numerically to ERS = 0.7C + 0.3A. This is documented as a limitation; separating exact-match scoring from topic coverage is recommended unless updated source code shows it has been fixed.

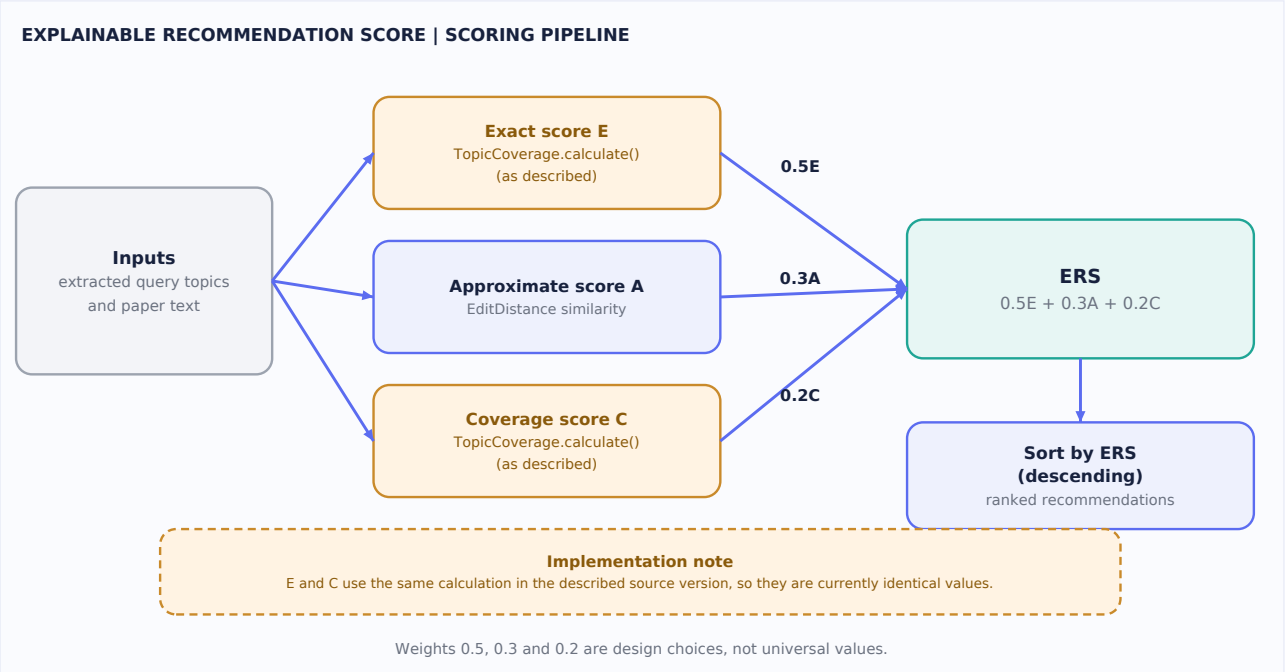


Figure 7. Scoring pipeline. Component scores are weighted, summed and used to sort the candidates.

ERS is a transparent heuristic, not a probability and not a measure of paper quality. Its usefulness depends on the dataset, the topic list and the weights, and it has not been evaluated against labelled relevance judgements. A worked numerical example, labelled hypothetical, appears in the Results chapter.

G. Bipartite graph and maximum matching

The topic–paper graph has two vertex sets: the extracted query topics and the candidate papers. An edge joins a topic to a paper when the recommendation's matched-topic list shows that the paper covers that topic. A matching is a set of edges in which no vertex appears twice, so each topic is assigned to at most one paper and each paper to at most one topic. A maximum matching has the largest possible number of edges [4], [6]. Its purpose in ETCM is to display distinct topic–paper assignments, so that a user can see which different papers can cover different topics.

The augmenting-path method builds a matching one topic at a time. For a topic, the algorithm tries each adjacent paper that has not been visited in this attempt. If the paper is unmatched it is assigned; if it is matched to another topic, the algorithm tries to move that topic to a different paper by a recursive depth-first search. If this succeeds the matching grows by one edge.

```
MAX-MATCHING(topics U, papers V, adjacency Adj)
  matchOf[v] <- NONE for every paper v
  size <- 0
  for each topic u in U do
      visited <- empty set
      if TRY(u) then size <- size + 1
  return matchOf, size

TRY(u)
  for each paper v in Adj[u] do
      if v not in visited then
          add v to visited
          if matchOf[v] = NONE or TRY(matchOf[v]) then
              matchOf[v] <- u
              return true
  return false
```

Figure 8 illustrates a hypothetical graph with three topics and three papers. The shaded edges form one maximum matching of size three. Several things are deliberately kept separate: the recommendation ranking and the coverage calculation come from the scoring pipeline, and the matching is only a one-to-one assignment over the resulting edges.

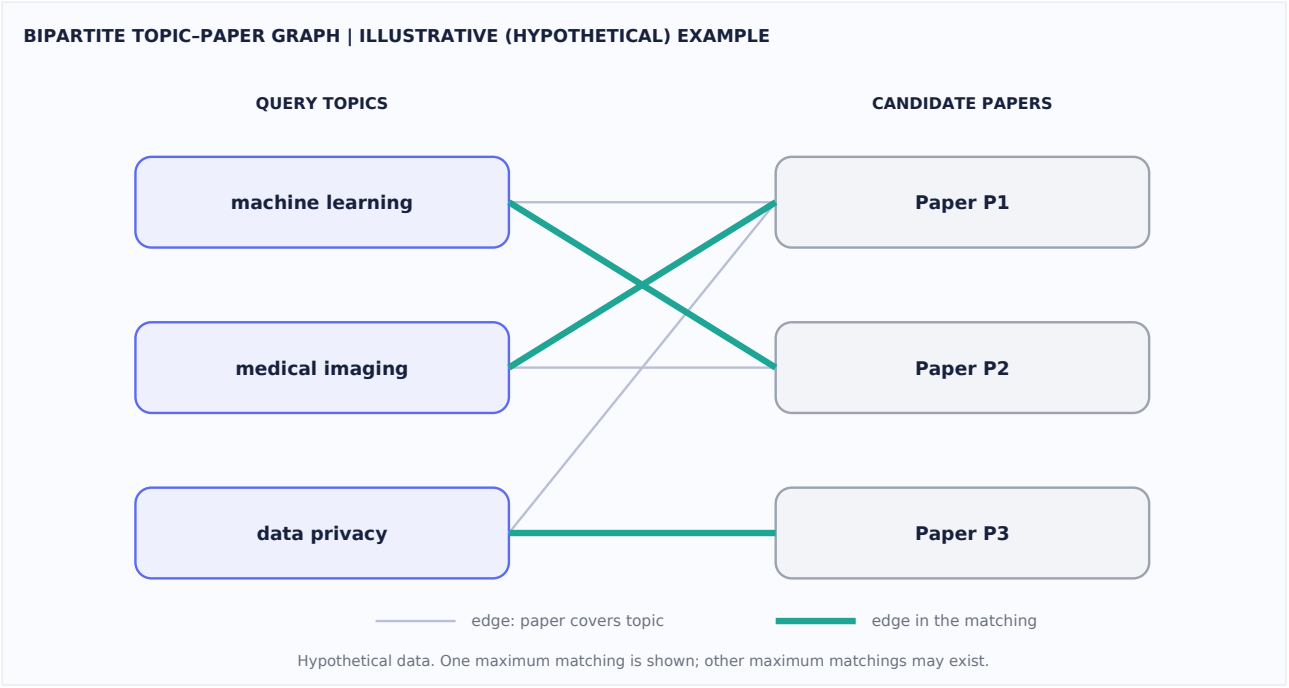


Figure 8. Illustrative topic–paper bipartite graph with one maximum matching highlighted (hypothetical data).

Boundaries of the matching: it is not a set-cover algorithm, and standard maximum bipartite matching does not maximise multi-topic coverage or ERS. A paper that covers all topics is still assigned to at most one topic in the matching.

For a graph with V vertices and E edges, a DFS-based augmenting-path implementation takes O(VE) time in the typical analysis, because each of at most V searches visits each edge at most once. The space needed is O(V + E).

Implementation details

Data loading. PaperCsvReader loads paper records from CSV into Paper objects holding ID, title, authors, keywords and abstract. A robust reader should handle a header row, quoted fields that contain commas, and incomplete rows; how the implementation handles each is to be verified and is covered by the test plan.

Scoring and ranking. RecommendationEngine retrieves candidates, computes the component scores for each, stores them with the matched topics in Recommendation objects and sorts by ERS in descending order. The treatment of equal scores is to be confirmed.

Interface and reports. ETCMApplication builds the JavaFX interface and BipartiteGraphView shows topics, papers, matching relationships and summary labels. The current view does not draw explicit graph edges; drawing them is listed as a future enhancement. ReportExporter writes CSV and text reports.

Prototype interface outputs

The four figures below are reserved for screenshots of the running application. No screenshot is invented: each frame is a labelled placeholder to be replaced with a real capture.

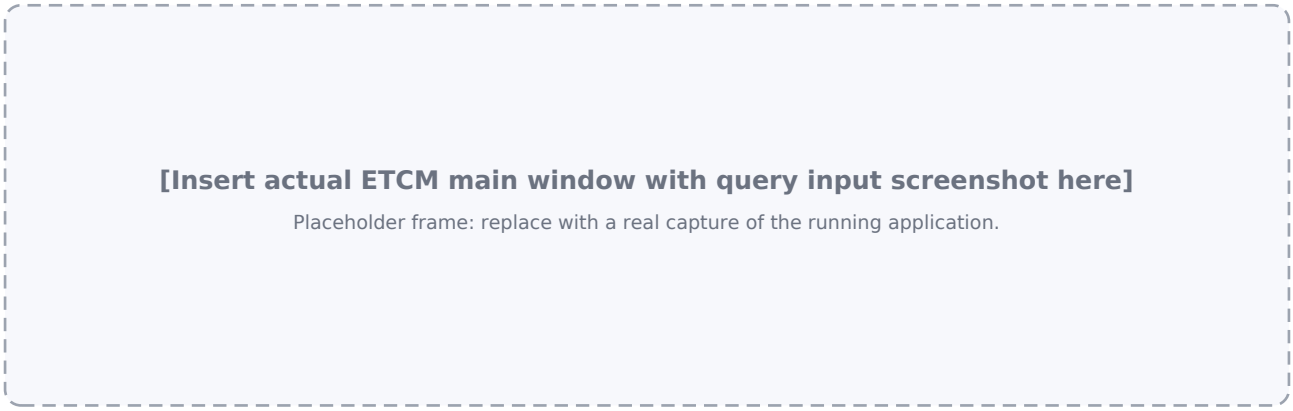


Figure 9. Main window showing the query input and dataset-loading controls. Placeholder: replace with an actual screenshot.

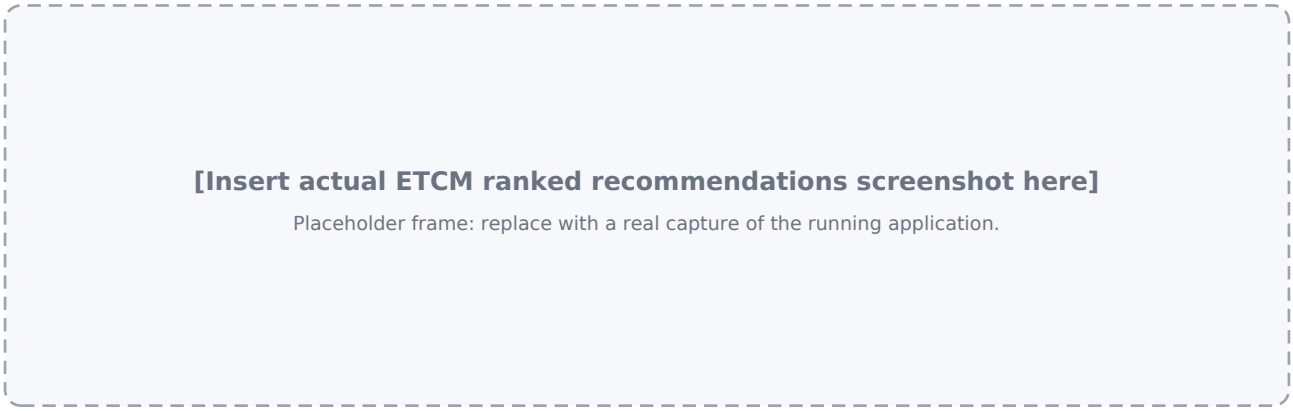


Figure 10. Recommendation results with ERS and matched topics. Placeholder: replace with an actual screenshot.

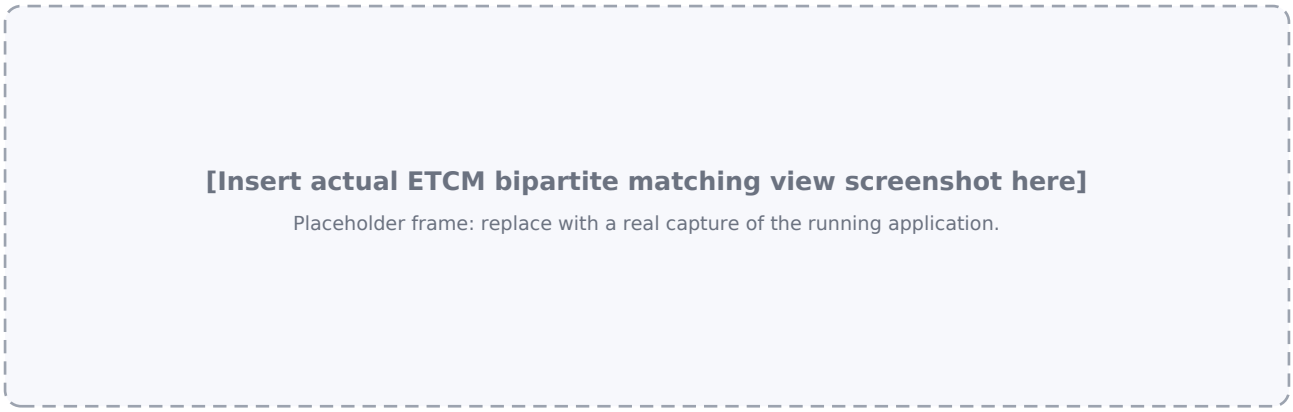


Figure 11. Topic–paper matching view with summary labels. Placeholder: replace with an actual screenshot.



Figure 12. An exported CSV file and text report. Placeholder: replace with an actual screenshot.

Complexity analysis

No single complexity figure describes the whole system, because the cost depends on several quantities. The variables used below are: **N**, the number of papers in the dataset; **T**, the total number of tokens in the indexed text; **q**, the number of distinct query terms; **c**, the number of candidate papers; **k**, the number of extracted query topics; **n**, the length of a paper text in characters; **m**, the length of a topic or pattern; **a** and **b**, the lengths of two strings in Edit Distance; and **V** and **E**, the vertices and edges of the bipartite graph, with $V = k + c$ and $E \leq kc$. Table 9 states time and space for each component with its assumptions.

Table 9. Complexity analysis

Component	Purpose	Time	Space	Assumptions
Inverted-index construction	Map terms to paper IDs	Expected O(T)	O(T) in the worst case (distinct term–paper pairs)	Hash operations take expected O(1) time
Candidate retrieval	Find papers sharing query terms	Expected O(q + total size of the q posting sets); at most O(qN)	O(c)	Union of posting sets assumed; to be confirmed
Rabin–Karp (one pattern, one text)	Exact string search	Expected O(n + m); worst case O(nm)	O(1)	Expected case assumes few spurious matches
Edit Distance (two strings)	Approximate similarity	O(ab)	O(ab) for the full table	Full table kept; a two-row version could use less space but is not claimed
Topic coverage (one paper)	Fraction of topics found	Expected O(k(n + m)) if each topic is searched in the text; worst case O(knm)	O(k)	Search method per topic to be confirmed
ERS scoring	Combine E, A and C	O(1) per paper, O(c) for all candidates	O(c)	Component scores already computed
Sorting	Rank by ERS	O(c log c) comparisons	O(c)	Comparison-based sort over c recommendations
Bipartite matching	Distinct topic–paper assignment	O(VE) with DFS augmenting paths	O(V + E)	Adjacency list representation assumed

Worst-case and average-case figures are kept apart on purpose. For example, the O(nm) worst case of Rabin–Karp applies only when collisions are frequent, and the expected O(T) index construction depends on hash behaviour. Because no timing has been measured, the report makes no claim about running time on any dataset size.

EXPERIMENTS

Experimental approach

This chapter is a formal test plan. The source code, execution logs and screenshots were not available while the report was prepared, so no test has been run by the author of this report and no outcome is recorded. Every actual-result field therefore reads "To be verified". The expected outcomes are derived by hand from the definitions in this report and can be checked against the program. No test is claimed to have passed, and no dataset size, timing or accuracy figure is stated.

Testing layers

Table 10 lists the layers at which ETCM should be tested.

Table 10. Testing layers

Layer	What to verify	Evidence to collect
Unit: text processing	Normalisation, stop words, phrase extraction, empty input	Test output for each input
Unit: algorithms	Rabin–Karp, Edit Distance, coverage, ERS arithmetic, sorting, matching	Return values compared with hand-derived results
Component: data input	CSV header, quoted fields, invalid rows	Loaded Paper objects and any error messages
Component: output	CSV and text report content	Exported files
Interface	Display of empty and non-empty results	Screenshots of the running application
Integration	Query to ranked, matched and exported results	End-to-end run record

Test cases

Each case has an identifier, purpose, input, expected outcome and actual result. Table 11 covers preprocessing and topic extraction, Table 12 covers the two string algorithms, Table 13 covers coverage, scoring, ranking and matching, and Table 14 covers CSV input, export and the interface.

Table 11. Test cases: preprocessing and topic extraction

ID	Purpose	Input	Expected outcome	Actual result
T-01	Lowercase normalisation	"Machine LEARNING"	"machine learning"	To be verified
T-02	Punctuation removal	"data-privacy, imaging!"	Punctuation replaced and spaces collapsed: "data privacy imaging" (replacement character to be confirmed)	To be verified
T-03	Stop-word filtering	"the use of machine learning in imaging"	Stop words such as "the", "of" and "in" removed; content words retained (exact stop list per source)	To be verified
T-04	Phrase extraction	"machine learning for medical imaging"	Predefined phrases "machine learning" and "medical imaging" extracted as topics	To be verified
T-05	Empty query	"" and " " (spaces only)	No topics; no recommendations; no exception; coverage is 0	To be verified

Table 12. Test cases: Rabin–Karp and Edit Distance

ID	Purpose	Input	Expected outcome	Actual result
T-06	Exact Rabin–Karp match	Text "the quick brown fox", pattern "quick"	Match reported at index 4	To be verified
T-07	No Rabin–Karp match	Text "the quick brown fox", pattern "slow"	No match reported (no-match convention to be confirmed)	To be verified
T-08	Pattern longer than text	Text "abc", pattern "abcdef"	No match and no exception	To be verified
T-09	Hash collision handled	Text "ju", pattern "ab" (both hash to 84 with BASE 256, PRIME 101)	Hashes agree but verification fails; no match reported	To be verified
T-10	Edit Distance, identical strings	"graph" and "graph"	Distance 0; similarity 1.0	To be verified
T-11	Edit Distance, one-character difference	"graph" and "grape"	Distance 1; similarity 0.8	To be verified
T-12	Edit Distance with an empty string	"" and "abc"	Distance 3; similarity 0.0 (both-empty case to be verified)	To be verified

Table 13. Test cases: coverage, scoring, ranking and matching

ID	Purpose	Input	Expected outcome	Actual result
T-13	Coverage 0	Topics {machine learning, medical imaging}; paper text contains neither	Coverage 0.0	To be verified
T-14	Partial coverage	Same topics; paper contains only "machine learning"	Coverage 0.5	To be verified
T-15	Full coverage	Same topics; paper contains both	Coverage 1.0	To be verified
T-16	No topics extracted	Empty topic list	Coverage 0 (no division by zero)	To be verified
T-17	ERS arithmetic	E = 0.8, A = 0.6, C = 0.5	ERS = 0.4 + 0.18 + 0.10 = 0.68	To be verified
T-18	Recommendation sorting	Papers with ERS 0.62, 0.91 and 0.75	Order 0.91, 0.75, 0.62 (treatment of ties to be confirmed)	To be verified
T-19	Bipartite matching with distinct assignments	Edges t1–P1, t1–P2, t2–P1	Matching size 2: t2–P1 and t1–P2 (requires an augmenting path); no paper used twice	To be verified
T-20	Empty matching graph	Topics and papers with no edges	Matching size 0; no exception	To be verified

Table 14. Test cases: CSV input, export and interface

ID	Purpose	Input	Expected outcome	Actual result
T-21	Valid CSV input	A CSV file with a header and several valid rows	All rows loaded as Paper objects with correct fields	To be verified
T-22	CSV headers	A file whose header is missing or differs from the expected header	Handled as defined by the reader (rejected, skipped or reported); no crash	To be verified
T-23	Quoted CSV fields	Row with a quoted title containing a comma	The quoted field stays one field	To be verified
T-24	Invalid or incomplete rows	Row with missing columns	Row skipped or reported; remaining rows still loaded	To be verified
T-25	Exported report content	Export after a query with results	CSV has a header and one row per recommendation with scores; text report lists query, topics, ranking and matched topics	To be verified
T-26	Interface with empty results	Query with no candidate papers	Empty-state display; no error dialog or crash	To be verified
T-27	Interface with non-empty results	Query with candidates	Ranking, matching information and summary labels shown	To be verified

Reproducibility and evidence

A repeatable run should load a small, known dataset (the sample format in Appendix B is suitable), run the cases above in order, and record for each case the input, the observed output, the date, the program version and the environment. A case that cannot be run should be marked blocked with the missing prerequisite, not passed. Expected values for the algorithm cases are small enough to verify by hand, which also helps in a viva.

Table 15 summarises execution status. The counts are cases proposed in this report.

Table 15. Test execution status

Group	Cases proposed	Results recorded	Status
Preprocessing and topic extraction (Table 11)	5	None	To be verified
Rabin–Karp and Edit Distance (Table 12)	7	None	To be verified
Coverage, scoring, ranking and matching (Table 13)	8	None	To be verified
CSV input, export and interface (Table 14)	7	None	To be verified
Total	**27**	**None**	**To be verified**

RESULTS

Implemented functional capabilities

Table 16 records the capabilities named in the project description. Because the source code, a run record and test logs were not available for this report, each is listed as described with its evidence status. A description of behaviour is not a measured result.

Table 16. Implemented capabilities as described and evidence status

Capability	Component	Behaviour (as described)	Evidence status
Topic extraction	TextPreprocessor	Normalisation, stop-word removal, phrase and word topics	Described; to be verified
Candidate retrieval	InvertedIndex	Term to paper-ID sets using HashMap and LinkedHashSet	Described; to be verified
Exact matching	RabinKarp	BASE = 256, PRIME = 101, verified matches	Described; to be verified
Approximate similarity	EditDistance	Dynamic-programming distance and normalised similarity	Described; to be verified
Coverage	TopicCoverage	Matched topics and coverage; 0 if no topics	Described; to be verified
Scoring and ranking	RecommendationEngine	ERS = 0.5E + 0.3A + 0.2C; descending sort; E and C identical	Described; limitation documented
Bipartite matching	BipartiteMatcher	Maximum matching by augmenting paths	Described; to be verified
Display	ETCMApplication, BipartiteGraphView	Recommendations, matching and summary labels	Described; screenshots pending
Data input and export	PaperCsvReader, ReportExporter	CSV load; CSV and text export	Described; to be verified

What is not claimed

- No benchmark timing, dataset size, accuracy value or user-study result is reported.
- No claim is made that ETCM outperforms other recommendation systems.
- ETCM is not described as an artificial-intelligence or deep-learning system; it uses deterministic classical algorithms.
- Maximum bipartite matching is not presented as set cover and is not claimed to maximise topic coverage or ERS.
- No claim is made that the ranking reflects scientific quality or that the project is experimentally validated.
- No screenshots, test results or faculty approvals are invented.

Hypothetical recommendation example

> **Everything in this section is hypothetical. The query, papers and values were constructed to illustrate the calculations. They are not results obtained from the ETCM program, and the approximate scores are assumed rather than computed by a specific formula.**

Query: "machine learning for medical imaging and data privacy". After normalisation and removal of the stop words "for" and "and", and assuming all three phrases are in the predefined list, the extracted topics are machine learning (ML), medical imaging (MI) and data privacy (DP). Three hypothetical candidate papers are used. Table 17 lists them with the topics each contains exactly.

Table 17. Hypothetical example: query topics and papers

Paper	Hypothetical title	Topics found exactly	Number found
P1	Privacy-Preserving Machine Learning for Medical Imaging	ML, MI, DP	3 of 3
P2	Machine Learning Methods for Medical Imaging Analysis	ML, MI	2 of 3
P3	A Survey of Data Privacy Regulations	DP	1 of 3

Coverage is the number found divided by the number of topics, so C = 1.000, 0.667 and 0.333. In the described version E equals C, so E takes the same values. The approximate scores are assumed as A = 0.95, 0.88 and 0.70. Table 18 applies ERS = 0.5E + 0.3A + 0.2C; for P2, 0.5(0.6667) + 0.3(0.88) + 0.2(0.6667) = 0.3333 + 0.2640 + 0.1333 = 0.7307.

Table 18. Hypothetical example: component scores, ERS and ranking

Rank	Paper	E	A	C	0.5E	0.3A	0.2C	ERS
1	P1	1.000	0.95	1.000	0.5000	0.2850	0.2000	**0.985**
2	P2	0.667	0.88	0.667	0.3333	0.2640	0.1333	**0.731**
3	P3	0.333	0.70	0.333	0.1667	0.2100	0.0667	**0.443**

The final ranking is P1 (0.985), P2 (0.731) and P3 (0.443). The matched-topic lists give the edges ML–P1, ML–P2, MI–P1, MI–P2, DP–P1 and DP–P3, which are the edges drawn in Figure 8. Processing the topics in the order ML, MI, DP with candidate papers in ranking order, the augmenting-path method proceeds as in Table 19.

Table 19. Hypothetical example: augmenting-path matching trace

Step	Topic processed	Action	Matching after the step
1	ML	P1 is free, so ML is assigned to P1	ML–P1
2	MI	P1 is taken by ML; ML is moved to the free paper P2, and MI takes P1	ML–P2, MI–P1
3	DP	P1 is taken by MI, and trying to move MI and then ML finds no unvisited paper, so that search fails; P3 is free	ML–P2, MI–P1, DP–P3

The final matching assigns ML to P2, MI to P1 and DP to P3, a maximum matching of size 3. It shows that the matching and the ranking are different things: P1 is ranked first and covers all three topics, yet in the matching it appears only once, against MI, because each paper can be used at most once. The matching gives distinct assignments for display; it does not change the ERS values or the ranking.

Advantages

- **Explainable scoring:** each score component and matched topic can be traced to a specific step.
- **Multiple classical techniques:** indexing, rolling hashing, dynamic programming, sorting and graph matching work together in one pipeline.
- **Transparent matching decisions:** the bipartite view shows which papers can cover which topics.
- **Modular implementation:** small classes with single responsibilities.
- **Local execution:** no network service or trained model is needed.
- **CSV-based extensibility:** a new dataset is a new file in the expected format.
- **Educational value:** a compact demonstration of string algorithms, hashing, dynamic programming, indexing and graph matching.

Results discussion and evaluation

The main outcome of the design is a recommendation process in which every decision can be explained. Candidate retrieval, exact matching, approximate similarity, coverage, weighting and matching are separate steps, and each can be tested with small inputs. The honest limitation is that the exact and coverage scores coincide in the described version, so the score has effectively two independent parts rather than three. Table 20 evaluates the project against its objectives.

Table 20. Evaluation against objectives

Objective	Assessment from the project description	Evidence still required
Candidate retrieval	Inverted index with HashMap and LinkedHashSet is described	Index tests; sample retrieval
Exact and approximate matching	Rabin–Karp and Edit Distance are described	Tests T-06 to T-12
Scoring and ranking	ERS and descending sort are described; E and C coincide	Tests T-13 to T-18; separation of E and C
Distinct topic–paper assignment	Maximum bipartite matching is described	Tests T-19 and T-20
Interface and export	JavaFX display and CSV and text export are described	Screenshots and tests T-25 to T-27
Effectiveness of recommendations	Not established	Labelled evaluation dataset and measured comparison

A fuller evaluation would need a labelled dataset and a comparison of scoring configurations, and measured retrieval times on defined dataset sizes. Until then the report describes how the system is designed to behave and not how well it performs.

CONCLUSION and FUTURE WORK

Ethical and academic considerations

- Recommendations depend on the supplied dataset; a paper missing from the CSV file cannot be recommended.
- A high score does not prove that a paper is correct or of high quality; ERS measures textual overlap with the query.
- Users should read the original publications before relying on them.
- Sample data and hypothetical examples in this report are identified as such and are not results of the program.
- Sources and references should be cited properly and any adapted material acknowledged.

Limitations and future enhancements

Table 21 separates the limitations of the current prototype from proposed enhancements. Proposed items are not implemented features.

Table 21. Limitations and future enhancements

Current limitation	Impact	Proposed enhancement (future work)
Heuristic topic extraction	Topics outside the phrase list appear only as words	Improve phrase extraction
Dependence on predefined technical phrases	New domains need list updates	Make the phrase list configurable and larger
Weak semantic understanding	Synonyms and related concepts are missed	Evaluate additional matching techniques separately
Potential false matches	Approximate similarity can link unrelated strings	Tune thresholds with a labelled dataset
Exact score and coverage score are identical	ERS has fewer independent parts than intended	Separate exact-match scoring from topic coverage
Fixed ERS weights	Weights may not suit every dataset	Add configurable scoring weights
One-to-one matching	A paper covering many topics is assigned to one	Add views that show all covered topics per paper
Graph view does not draw explicit edges	Relationships are harder to read	Draw explicit graph edges in the interface
Modest local datasets	Scalability is untested	Support larger datasets and benchmark on measured data
Simple CSV handling	Unusual files may load incorrectly	Use a more robust CSV parser
No guarantee of scientific relevance	Users must judge quality themselves	Add metadata and filtering options

Conclusion

ETCM demonstrates that a research paper recommendation prototype can be built from classical data structures and algorithms and still explain its decisions. An inverted index narrows the collection, Rabin–Karp and Edit Distance provide exact and approximate string evidence, topic coverage shows how much of a multi-topic query a paper addresses, a weighted score ranks the candidates, and a bipartite matching displays distinct topic–paper assignments. Each step uses a technique that can be traced, analysed and tested.

The prototype has clear limits. Topic extraction is heuristic, matching is textual rather than semantic, the exact and coverage scores currently coincide, and the matching does not improve the ranking. No claim is made of scientific superiority, production readiness or proven effectiveness. The value of the

project is as a coherent, explainable demonstration of the course material, which should next be supported by measured tests.

The next step is to run the test plan, separate the exact-match score from topic coverage, and evaluate scoring configurations on a labelled dataset.

References

[1] R. M. Karp and M. O. Rabin, "Efficient randomized pattern-matching algorithms," IBM Journal of Research and Development, vol. 31, no. 2, 1987. (Verify volume and page numbers before submission.)

[2] V. I. Levenshtein, "Binary codes capable of correcting deletions, insertions, and reversals," Soviet Physics Doklady, vol. 10, no. 8, 1966. (Verify pages before submission.)

[3] R. A. Wagner and M. J. Fischer, "The string-to-string correction problem," Journal of the ACM, vol. 21, no. 1, 1974. (Verify pages before submission.)

[4] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 4th ed. MIT Press, 2022.

[5] C. D. Manning, P. Raghavan and H. Schütze, Introduction to Information Retrieval. Cambridge University Press, 2008. https://nlp.stanford.edu/IR-book/

[6] H. W. Kuhn, "The Hungarian method for the assignment problem," Naval Research Logistics Quarterly, vol. 2, 1955. (Verify issue and pages before submission.)

[7] Oracle, "Java Platform, Standard Edition 21 Documentation." https://docs.oracle.com/en/java/javase/21/

[8] OpenJFX, "JavaFX Documentation." https://openjfx.io/

[9] Oracle, "The Java Tutorials: Collections." https://docs.oracle.com/javase/tutorial/collections/

[10] Eclipse Foundation, "Eclipse IDE." https://www.eclipse.org/ide/

[11] R. Sedgewick and K. Wayne, Algorithms, 4th ed. Addison-Wesley, 2011.

[12] D. Jurafsky and J. H. Martin, Speech and Language Processing (online draft, chapter on edit distance). https://web.stanford.edu/~jurafsky/slp3/ (Verify the edition and date used.)

[13] Project description for ETCM, provided by the project team for this report. The source code was not supplied.

Source and evidence note

Project statements come from the project description supplied by the team. Examples marked hypothetical are illustrations. Reference entries marked for verification must have their volume, pages or edition confirmed against the original before submission, and web links should be re-checked.

Technical source map

Table 22 maps each topic to the artefact that should hold the corresponding evidence. File names are given only for the classes listed in the project description; line-level references are to be added from the repository.

Table 22. Technical source map

Topic	Primary project artefact
Preprocessing and topic extraction	TextPreprocessor.java
Inverted index	InvertedIndex.java, Paper.java
Rabin–Karp	RabinKarp.java
Edit Distance	EditDistance.java
Topic coverage	TopicCoverage.java
Scoring and ranking	RecommendationEngine.java, Recommendation.java
Bipartite matching	BipartiteMatcher.java
Interface	ETCMApplication.java, BipartiteGraphView.java, Main.java
Data input and export	PaperCsvReader.java, ReportExporter.java
Tests	To be added: test classes or the evidence table described in the Experiments chapter

Table 23 lists the points to confirm before submission.

Table 23. Items requiring verification before submission

Item	What to confirm
Cover page details	University, department, guide, course, academic year and submission date placeholders
Topic extraction	Whether single words inside phrases are kept; the stop-word list; the phrase list
Retrieval	Which paper fields are indexed; union or intersection of posting sets
Approximate score A	How the paper-level approximate score is aggregated from Edit Distance similarities
Empty strings	Edit Distance similarity when both strings are empty; Rabin–Karp with an empty pattern
Sorting	Handling of equal ERS values
E and C	Whether the identical-calculation issue is still present in the final source
CSV reader	Header names, quoted-field handling and treatment of incomplete rows
Screenshots and tests	Replace placeholders and run the test plan

Appendix A: Pseudocode

Pseudocode for Rabin–Karp, Edit Distance and the augmenting-path matching appears in the Methodology chapter. The remaining procedures are given here and follow the definitions in this report.

Text preprocessing and topic extraction

```
PREPROCESS(text)
  text <- lowercase(text)
  text <- replace every punctuation character by a space
  text <- collapse runs of whitespace to one space and trim
  return text

EXTRACT-TOPICS(text, phraseList, stopWords)
  words <- split(PREPROCESS(text)) without stop words
  topics <- empty ordered set
  for each phrase in phraseList do
      if phrase occurs in the cleaned text then add phrase to topics
  for each word in words not covered by a recognised phrase do   (single-word policy: to be confirmed)
      add word to topics
  return topics
```

Inverted index

```
BUILD-INDEX(papers)
  index <- empty HashMap
  for each paper in papers do
      for each term in terms(paper) do                (fields indexed: to be confirmed)
          index[term] <- index[term] with paper.id added   (LinkedHashSet)
  return index

CANDIDATES(index, queryTerms)
  result <- empty LinkedHashSet
  for each term in queryTerms do
      if term in index then add all of index[term] to result
  return result
```

Topic coverage, ERS and ranking

```
COVERAGE(topics, paperText)
  if topics is empty then return 0
  found <- topics that occur in the normalised paperText
  return |found| / |topics|

ERS(E, A, C)
  return 0.5*E + 0.3*A + 0.2*C

RANK(candidates)
  for each paper in candidates do compute E, A, C and ERS; store with its matched topics
  sort recommendations by ERS in descending order      (ties: to be confirmed)
  return the sorted list
```

Appendix B: Sample dataset format

ETCM loads paper metadata from CSV files. The fields below follow the paper attributes in the project description (ID, title, authors, keywords and abstract). The column names and order are illustrative and must be matched to PaperCsvReader. Table 24 describes the fields, and the sample rows that follow are fictional.

Table 24. Illustrative paper CSV fields

Field	Meaning	Notes
paper_id	Unique paper identifier, such as P001	Used as the key in the inverted index
title	Paper title	May contain commas; quote the field
authors	Author list	May contain commas or semicolons; quote the field
keywords	Keyword list	May contain several terms in one field
abstract	Paper abstract	Long text; quote the field and double any internal quotation mark

```
paper_id,title,authors,keywords,abstract
P001,"Sample Paper A","A. Author","machine learning; data privacy","Fictional abstract text."
P002,"Sample Paper B","B. Author","learning; methods","Fictional abstract text."
P003,"Sample Paper C","C. Author","medical imaging","Fictional abstract text."
```

The papers above are fictional and exist only to show the format. Test files should avoid real personal or copyrighted text that is not meant to be shared.

Appendix C: User manual

The steps below describe the intended use. Control names and menu labels are not given because they must be taken from the application; replace the bracketed items with the real labels.

- **Prerequisites:** Java 21, JavaFX 21.0.12 and the Eclipse IDE [7], [8], [10].
- **Set up the project:** import the project into Eclipse and add the JavaFX libraries. When launching from an IDE, JavaFX normally needs VM arguments such as --module-path "<path to JavaFX lib>" and --add-modules javafx.controls; confirm the arguments the project requires.
- **Prepare a dataset:** create a CSV file in the format of Appendix B.
- **Run:** run Main.java. The application window opens (see the main-window screenshot placeholder).
- **Load papers:** use [dataset control] to select the CSV file and wait for loading to complete.
- **Enter a query:** type a research query or supply paper text using [input control].
- **Run the recommendation:** use [recommend control]. The ranked recommendations, matched topics, matching view and summary labels appear.
- **Read the results:** a higher ERS means more topic coverage and closer string similarity under the configured weights; it does not mean the paper is better.
- **Export:** use [export control] to save the CSV and text reports.
- **Troubleshooting:** if the window does not open, check the JavaFX module path; if no papers load, check the CSV header and quoting; if no recommendations appear, check that the query contains topics present in the dataset.

Appendix D: Viva questions and answers

Q. What does the inverted index store and why is it useful?

A. It maps each term to the set of paper IDs that contain it. It lets the system find candidates for a query without scanning every paper. It does not guarantee semantic relevance.

Q. Why are a HashMap and a LinkedHashSet used?

A. The HashMap gives expected constant-time lookup and insertion by term. The LinkedHashSet stores each ID once and keeps insertion order, so output is deterministic.

Q. How does Rabin–Karp work, and what are BASE and PRIME here?

A. It compares the pattern hash with a rolling hash of each window and verifies characters only when hashes match. BASE is 256 and PRIME is 101.

Q. What happens on a hash collision?

A. Two different strings share a hash, so the equal hashes trigger a character-by-character check, which fails. Correctness is kept; only extra work is added. "ab" and "ju" both hash to 84.

Q. What are the time complexities of Rabin–Karp?

A. Expected O(n + m) under favourable assumptions and worst case O(nm), where n is the text length and m the pattern length.

Q. Explain the Edit Distance recurrence.

A. D[i][j] is the minimum of deletion (D[i−1][j] + 1), insertion (D[i][j−1] + 1) and replacement or match (D[i−1][j−1] + cost), with cost 0 if the characters match and 1 otherwise.

Q. What is the Edit Distance of "graph" and "grape"?

A. One, by replacing h with e. The normalised similarity is 1 − 1/5 = 0.8.

Q. How is topic coverage defined?

A. The number of extracted query topics found in the paper divided by the total number of extracted topics; it is 0 if no topics were extracted.

Q. What is ERS and how were the weights chosen?

A. ERS = 0.5E + 0.3A + 0.2C. The weights are design choices, not established universal values, and could be tuned with a labelled dataset.

Q. What is the known issue with E and C?

A. In the described source version both come from TopicCoverage.calculate(), so they are identical and ERS equals 0.7C + 0.3A. The recommended fix is to separate exact-match scoring from coverage.

Q. Is the bipartite matching a set-cover algorithm?

A. No. It is a one-to-one assignment between topics and papers. It does not maximise coverage or ERS, and ranking and coverage are computed separately.

Q. What is the complexity of the matching?

A. O(VE) for the DFS-based augmenting-path method, with V vertices and E edges, and O(V + E) space.

Q. Does ETCM use machine learning?

A. No. It uses deterministic classical algorithms and no trained model. It also makes no claim to outperform other systems without experimental evidence.